

Software Design Specification

Handwritten Digit Recognition using Convolutional Neural Network (CNN)

Prepared By	Mayuresh Khedkar (EN23CS301608)
Guided By	Dr. Garima Silakari Tukra & Dr. Sheetal Bawane
Institution	Medicaps University, Indore – 453331
Department	Computer Science & Engineering
Programme	Bachelor of Technology in Computer Science & Engineering
Version	1.0
Date	January – May 2026

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Document Conventions
 - 1.3 Intended Audience and Reading Suggestions
 - 1.4 References
- 2. Analysis Model
 - 2.1 Methodology Used
 - 2.2 Use Case Diagram
 - 2.3 Activity Diagram
 - 2.4 Sequence Diagram
 - 2.5 Class Diagram
- 3. Design Model
 - 3.1 Architectural Design
 - 3.2 Database Design
 - 3.3 Component Design
 - 3.4 Interface Design
- Appendix C1: Glossary
- Appendix C2: To Be Determined List

1. Introduction

1.1 Purpose

This Software Design Specification (SDS) document describes the complete software design for the Handwritten Digit Recognition System (Version 1.0) developed as a Minor Project for the Bachelor of Technology in Computer Science & Engineering programme at Medicaps University, Indore.

The system employs a Convolutional Neural Network (CNN) model built with TensorFlow/Keras and trained on the MNIST dataset to classify handwritten digits (0–9) with high accuracy. It is deployed as a Flask-based web application through which users can upload an image and receive a real-time prediction.

This document covers the full design scope of the system — including analysis models (use case, activity, sequence, and class diagrams), architectural design, database design, component design (package, component, and deployment diagrams), and interface design. It serves as a reference for development, testing, and evaluation.

1.2 Document Conventions

The following conventions are used throughout this document:

- All section headings are formatted in bold using a hierarchical numbering scheme (e.g., 1.1, 1.2, 2.1).
- Technical terms such as class names, method names, and file names are written in monospace font (e.g., `app.py`, `predict()`).
- Diagrams that cannot be rendered graphically are described textually in a structured, formal manner.
- Higher-level requirements defined in the SRS are referenced by their identifier (e.g., REQ-1 through REQ-8) and are inherited by detailed design specifications in this document.
- Acronyms and abbreviations are defined upon first use and also listed in Appendix C1.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following readers:

- Developers: Should read all sections in order, with special attention to Sections 2 (Analysis Model) and 3 (Design Model) for implementation guidance.
- Project Evaluators / Professors: Should review Section 1 (Introduction) followed by Sections 3.1 (Architectural Design) and 3.4 (Interface Design) for a high-level overview.
- Testers: Should focus on Section 2 (Use Case and Activity Diagrams) to understand system behavior and expected flows.
- Students: May read the document sequentially for a comprehensive understanding of how an ML-powered web application is designed.

Recommended reading sequence: Introduction → Analysis Model → Design Model → Appendices.

1.4 References

The following documents and sources are referenced in this SDS:

- LeCun Y., Bottou L., Bengio Y., Haffner P., 'Gradient-based learning applied to document recognition,' Proceedings of the IEEE, Vol. 86, Issue 11, pp. 2278–2324, 1998.
- Simard P.Y., Steinkraus D., Platt J.C., 'Best practices for CNNs applied to visual document analysis,' ICDAR, pp. 958–963, 2003.
- Goodfellow I., Bengio Y., Courville A., Deep Learning, MIT Press, pp. 326–366, 2016.
- TensorFlow Documentation: https://www.tensorflow.org/api_docs
- Flask Documentation: <https://flask.palletsprojects.com/>
- MNIST Dataset: <http://yann.lecun.com/exdb/mnist/>

2. Analysis Model

2.1 Methodology Used

Programming Approach: Object-Oriented Programming (OOP)

The system is developed using an Object-Oriented Programming approach. Python, which supports full OOP paradigms, is used throughout the project. Key components are encapsulated as classes and modules:

- The CNN model is encapsulated within the TensorFlow/Keras Model class, with convolution, pooling, and dense layers represented as objects.
- The Flask application is structured as a Python module with route handlers acting as methods corresponding to defined system behaviors (e.g., image upload, prediction).
- Image preprocessing is handled by a dedicated utility class interfacing with PIL and NumPy.

Software Process Model: Iterative and Incremental Development

The project follows an Iterative and Incremental development methodology, which is well-suited for machine learning projects where performance is refined over successive iterations. The development is divided into the following phases:

- Iteration 1 – Requirements and Research: Problem definition, dataset exploration, and technology stack finalization.
- Iteration 2 – Model Development: CNN architecture design, training on MNIST, and accuracy evaluation.
- Iteration 3 – Backend Integration: Flask API setup, model loading, and image preprocessing pipeline.
- Iteration 4 – Frontend Development: HTML/CSS interface for image upload and result display.
- Iteration 5 – Testing and Optimization: End-to-end testing, error handling, and performance tuning.
- Iteration 6 – Documentation and Finalization: SRS, SDS, project report, and demonstration preparation.

2.2 Use Case Diagram

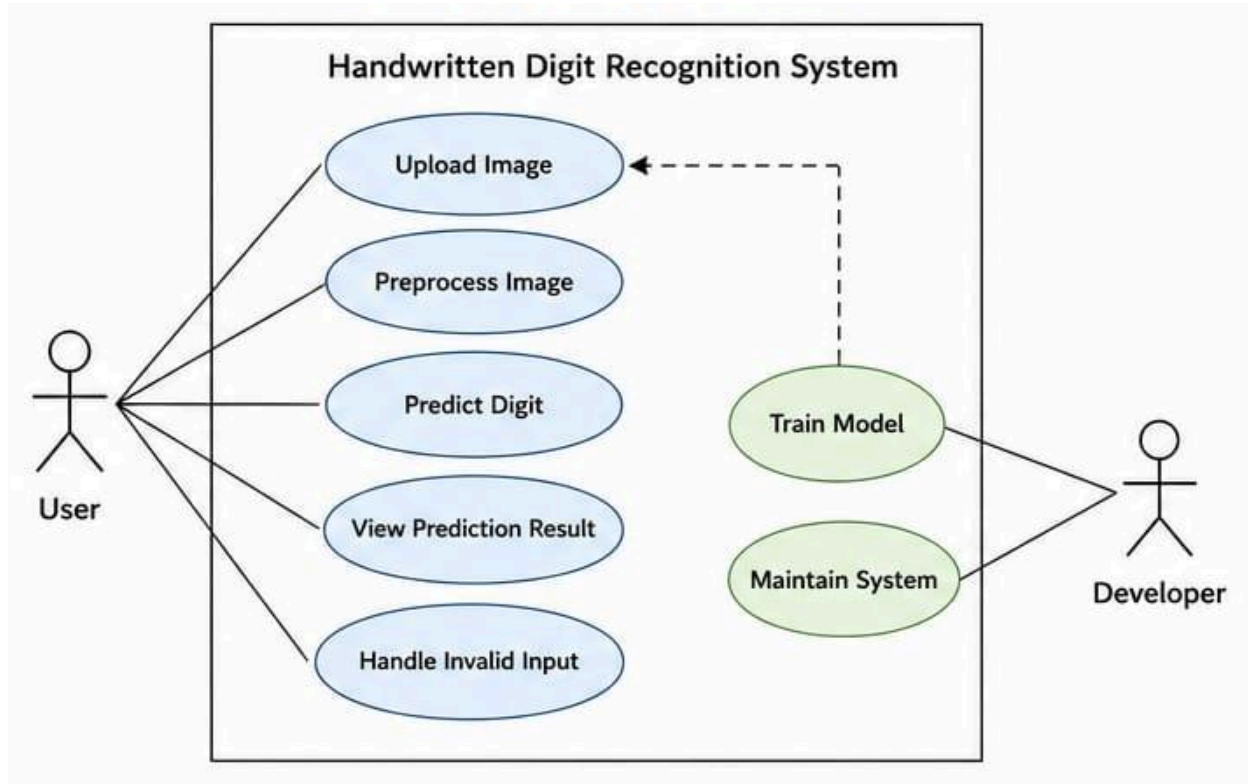
Use Case Diagram Description

The Use Case Diagram for the Handwritten Digit Recognition System identifies the following use cases:

Use Cases:

- UC-1: Upload Handwritten Digit Image – The user selects and submits an image file via the web interface.
- UC-2: Validate Image Input – The system checks whether the uploaded file is a valid image in an accepted format.
- UC-3: Preprocess Image – The system converts the image to grayscale, resizes it to 28x28 pixels, and normalizes pixel values.
- UC-4: Predict Digit – The preprocessed image is passed through the CNN model to produce a class prediction (0–9).

- UC-5: Display Prediction Result – The predicted digit is displayed on the webpage for the user.
- UC-6: Handle Invalid Input – If the uploaded file is invalid, an appropriate error message is displayed.



Use Case Specification: UC-1 – Upload Handwritten Digit Image

Field	Details
Name of Use Case	UC-1: Upload Handwritten Digit Image
Actor	User (Primary)
Pre Condition	The web application is running and accessible in a browser. The user has a handwritten digit image file available on their device.
Primary Flow of Events	1. User navigates to the web application home page. 2. User clicks the 'Choose File' button. 3. User selects a valid image file (PNG, JPG, or JPEG) from local storage. 4. User clicks the 'Predict' button. 5. System validates the uploaded file. 6. System preprocesses the image (grayscale conversion, resizing, normalization). 7. System passes the preprocessed image to the CNN model. 8. Model returns the predicted digit. 9. System displays the predicted digit on the result section of the webpage.
Alternate Flow 1	If the user does not select a file and clicks 'Predict', the system displays a prompt: 'Please select an image file.'

Alternate Flow 2	If the uploaded file is not a valid image format, the system displays an error message: 'Invalid file type. Please upload a PNG or JPG image.'
Post Condition	The predicted digit (0–9) is displayed on the web interface. The uploaded image is not stored permanently on the server.
Use Case Termination	The use case terminates when the prediction result is successfully displayed, or when an error message is shown to the user upon invalid input.

Use Case Specification: UC-4 – Predict Digit

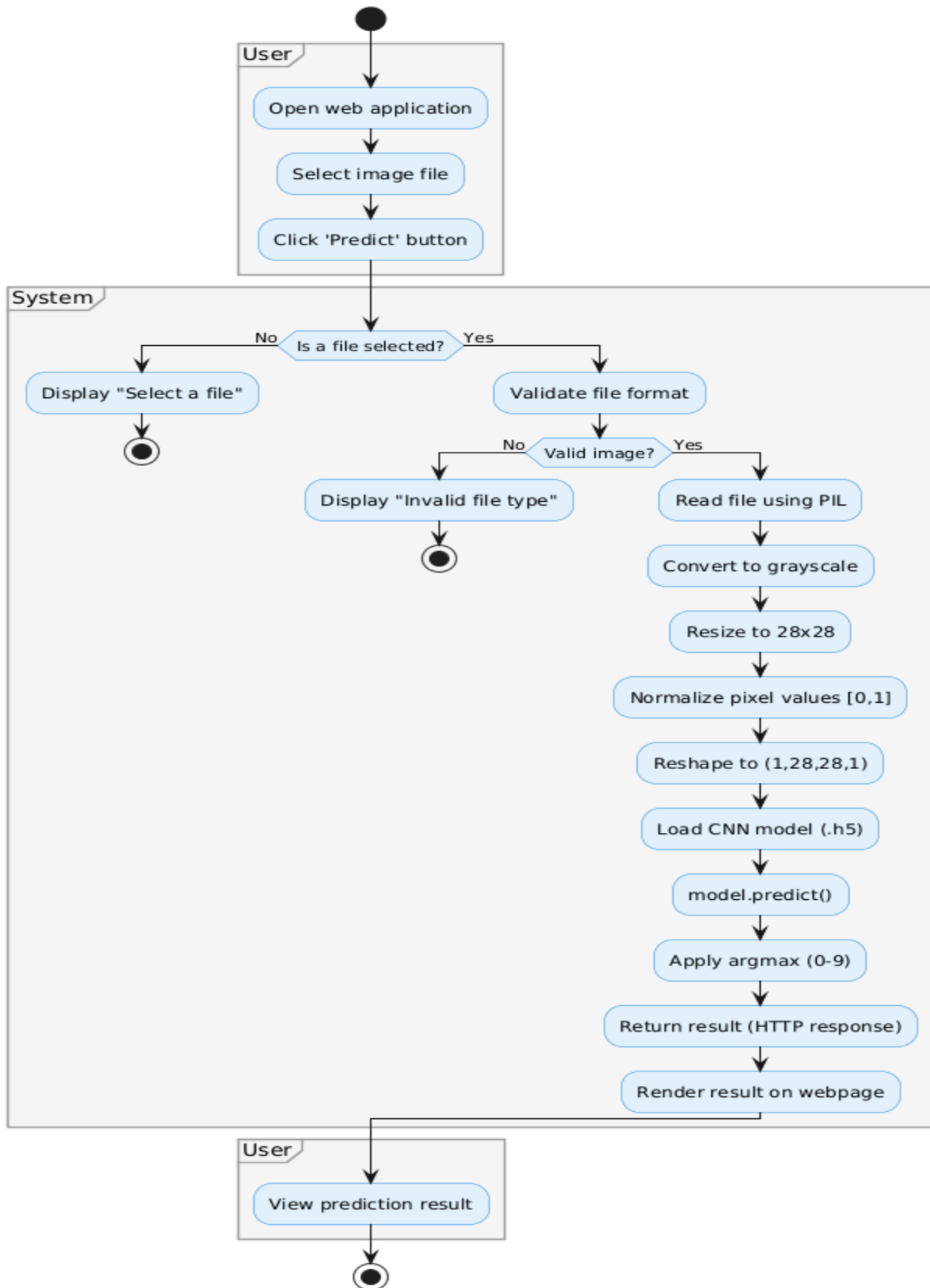
Field	Details
Name of Use Case	UC-4: Predict Digit
Actor	CNN Model (System Actor)
Pre Condition	The image has been successfully preprocessed into a 28x28 grayscale normalized NumPy array. The trained CNN model (.h5 file) is loaded in memory.
Primary Flow of Events	1. The preprocessed image array is passed to the model's predict() method.2. Model outputs a probability distribution across 10 classes (digits 0–9).3. The class with the highest probability is selected as the predicted digit.4. The predicted digit value is returned to the Flask route handler.
Alternate Flow 1	If the model fails to load (file not found or corrupt), the system logs the error and returns an HTTP 500 Internal Server Error with an appropriate message.
Alternate Flow 2	If prediction confidence for all classes is below a defined threshold, the system may optionally flag the result as 'Low Confidence' (planned for future enhancement).
Post Condition	A single integer digit (0–9) is returned as the prediction result.
Use Case Termination	Use case terminates when the predicted digit is successfully passed to the response handler for display.

2.3 Activity Diagram

Activity Diagram Description – Image Upload and Prediction Flow

The Activity Diagram describes the step-by-step workflow of the Handwritten Digit Recognition system from user interaction through prediction display. The diagram captures two swimlanes: User and System.

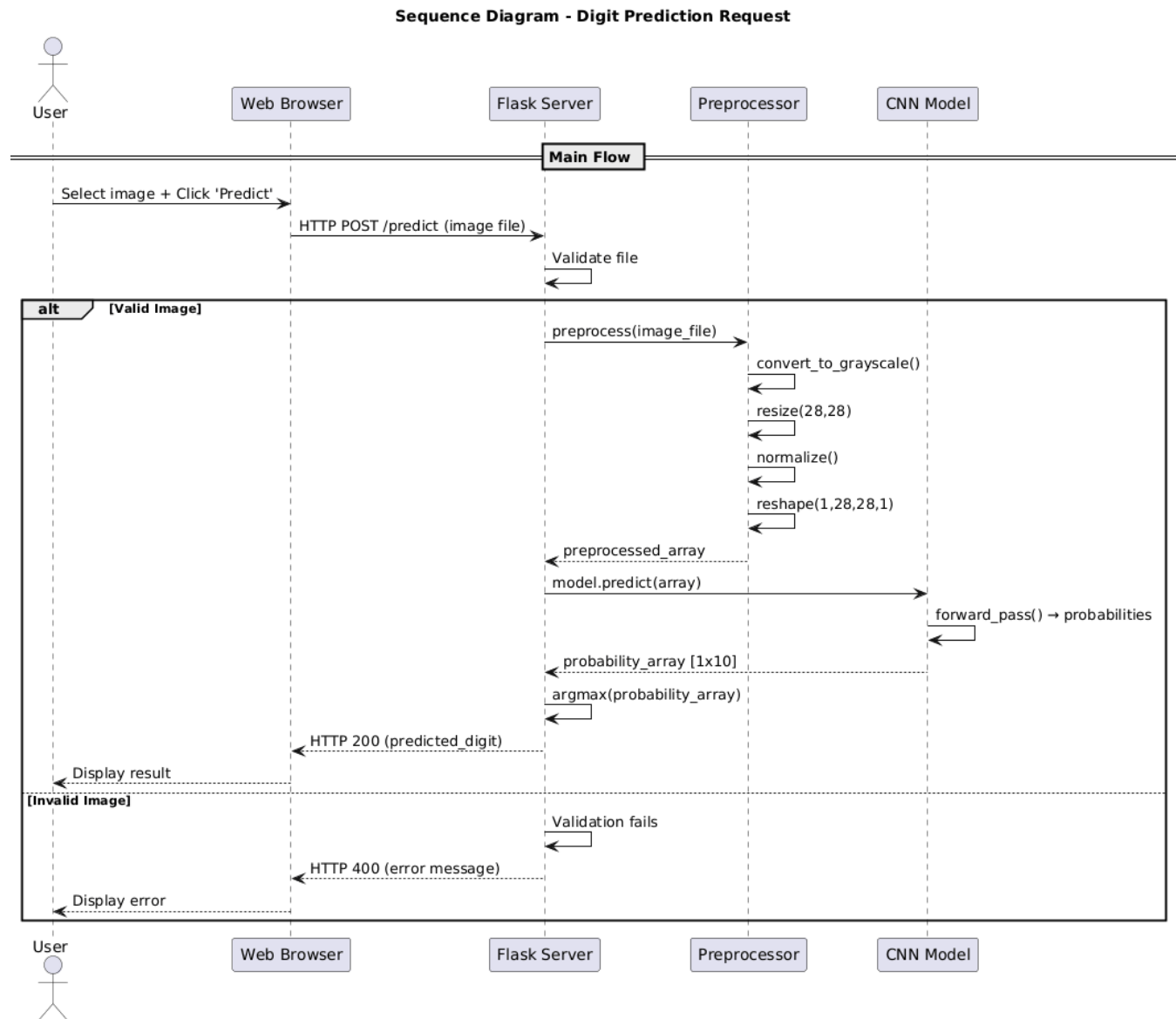
Activity Diagram - Image Upload and Prediction Flow



2.4 Sequence Diagram

Sequence Diagram Description – Digit Prediction Request

The Sequence Diagram illustrates the temporal ordering of interactions between objects during a digit prediction request. The participating objects/lifelines are: User, Web Browser, Flask Server, Preprocessor, CNN Model.



2.5 Class Diagram

Class Diagram Description

The Class Diagram reflects the actual implementation of the system, which is structured as a single-file Flask backend (app.py). All logic — including image preprocessing, model inference, and request handling — is contained within this one module. There are no separate class files; instead, the design is represented through two logically distinct functional units: the Flask Application module and the preprocess_image() utility function. The class diagram below maps directly to the code as written.

Module / Logical Class: FlaskApp (app.py)

- Description: The main application module. Initialises the Flask web server, enables CORS, loads the pre-trained CNN model at startup, and defines the index() route handler that serves the upload form and processes POST requests.
- Module-level attributes (globals): app: Flask, model: keras.Model (loaded from “8epochs.h5” at startup)
- Route / Method: index(request) — handles GET (renders upload page) and POST (receives uploaded image, calls preprocess_image(), calls model.predict(), resolves argmax, passes prediction to template)
- Dependencies (imports): Flask, flask_cors.CORS, tensorflow.keras, numpy, PIL.Image, PIL.ImageOps

Function: preprocess_image(image: PIL.Image) — defined in app.py

- Description: A standalone utility function (not a class method) that encapsulates the complete image preprocessing pipeline. It is called by index() each time a POST request is received.
- Input: image — a PIL.Image object opened directly from the uploaded file stream.
- Output: numpy.ndarray of shape (1, 28, 28, 1), dtype float32, values in range [0.0, 1.0].
- Internal steps (in order as coded): image.convert(“L”) → image.resize((28, 28)) → ImageOps.invert(image) → np.array(image) → image / 255.0 → image.reshape(1, 28, 28, 1)

External Entity: keras.Model (loaded from 8epochs.h5)

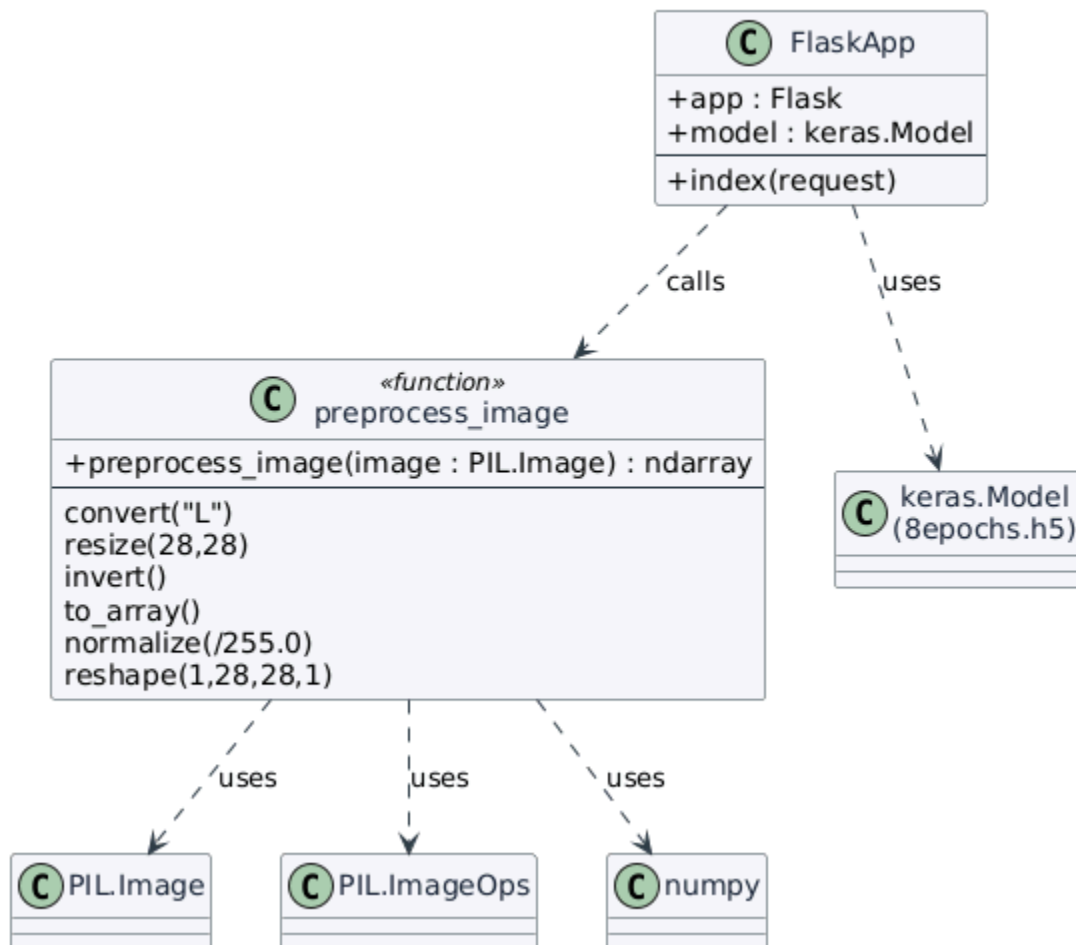
- Description: A pre-trained TensorFlow/Keras Sequential CNN model. Not defined in app.py; loaded as an external artefact via tf.keras.models.load_model() at application startup and held in the module-level global variable model.
- Used via: model.predict(preprocessed_array) — returns a numpy.ndarray of shape (1, 10) representing the probability distribution across digit classes 0–9.
- Predicted class resolved by: numpy.argmax(preds), which returns a single integer in the range [0, 9].

Relationships (as they exist in the code):

- FlaskApp calls preprocess_image() — a direct function call dependency within the same module. The index() route invokes preprocess_image(image) and uses the returned ndarray for inference.
- FlaskApp uses keras.Model — the global model object is loaded once at startup and accessed directly inside index() for prediction. This is a module-level dependency (usage association).

- `preprocess_image()` uses `PIL.Image` and `PIL.ImageOps` — library dependencies for grayscale conversion, resizing, and colour inversion.
- `preprocess_image()` uses `numpy` — for array conversion, normalisation ($/ 255.0$), and reshaping to $(1, 28, 28, 1)$.
- Note: There is no separate `InputValidator` class or `PredictionResult` class in the actual implementation. Input is read directly from `request.files["image"]` and the prediction integer is passed inline to `render_template()`. This flat, single-module design is intentional for a lightweight academic prototype.

Class Diagram - Flask-Based Digit Recognition System



3. Design Model

3.1 Architectural Design

3.1.1 System Architectural Diagram

Architecture Description (Client-Server, Three-Tier Architecture)

The system follows a three-tier client-server architectural pattern:

Tier 1 – Presentation Layer (Frontend):

- Technology: HTML5, CSS3
- Components: Web Browser, Image Upload Form, Prediction Result Display Section
- Interaction: User submits an image via an HTML form; the browser renders the returned prediction result.

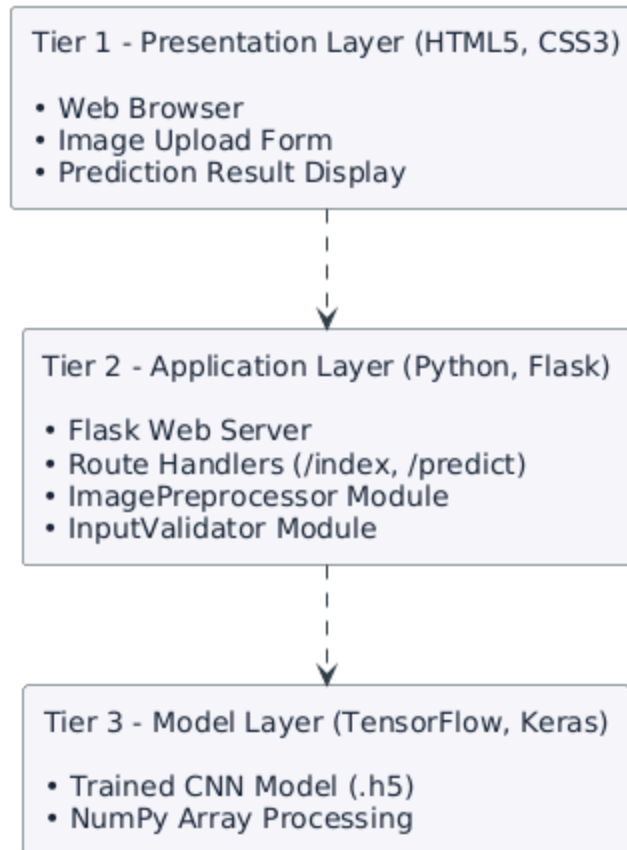
Tier 2 – Application Layer (Backend):

- Technology: Python, Flask Framework
- Components: Flask Web Server, Route Handlers (/index, /predict), ImagePreprocessor Module, InputValidator Module
- Interaction: Receives HTTP POST requests from the browser, processes the image, invokes the CNN model, and returns the response.

Tier 3 – Model Layer (AI/ML Core):

- Technology: TensorFlow, Keras
- Components: Trained CNN Model (.h5 file), NumPy Array Processing
- Interaction: Receives a preprocessed 28×28×1 NumPy array; returns a probability distribution over 10 digit classes.

3-Tier Architecture - Handwritten Digit Recognition System



System Architectural Diagram – Textual Representation:

Layer	Component	Technology	Communication
Presentation Layer	Web Browser (Client)	HTML, CSS	HTTP (GET/POST)
Application Layer	Flask Web Server	Python, Flask	Function Calls
Application Layer	Image Preprocessor	PIL, NumPy	In-Process
Application Layer	Input Validator	Python	In-Process
Model Layer	CNN Prediction Engine	TensorFlow/Keras	In-Process API
Model Layer	Saved Model File	.h5 (HDF5 Format)	File I/O (model.load)

3.1.2 Description of Architectural Diagram

The architectural design of the Handwritten Digit Recognition System is based on the Model-View-Controller (MVC) pattern realized through Flask's routing mechanism:

- **View (Frontend):** The HTML/CSS-based web interface serves as the View layer. It renders the user interface, provides the image upload form, and displays the prediction result. The View communicates exclusively through HTTP requests to the Flask backend.
- **Controller (Flask Routes):** The Flask application acts as the Controller. The `/index` route serves the home page, while the `/predict` route handles POST requests. The controller coordinates between the `InputValidator`, `ImagePreprocessor`, and `CNNModel` to orchestrate the prediction workflow.
- **Model (CNN + Data Logic):** The CNN model loaded from the `.h5` file constitutes the Model layer. It is loaded once at application startup and retained in memory for fast inference. The `ImagePreprocessor` transforms raw image uploads into the exact tensor format expected by the model (shape: 1, 28, 28, 1; dtype: float32; range: 0.0 to 1.0).
- **Data Flow:** User uploads image → Browser sends HTTP POST → Flask validates and preprocesses image → CNN Model performs inference → Prediction result returned as HTTP response → Browser displays result.
- **Scalability Consideration:** The stateless design of the Flask backend means the system can be scaled horizontally by running multiple Flask instances behind a load balancer, with the model file shared across instances.

3.2 Database Design

3.2.1 Data Dictionary

The Handwritten Digit Recognition System does not use a persistent relational database, as no user data or prediction history is stored between sessions. However, the following data dictionary defines the key data elements that are processed in-memory during each prediction request:

Field Name	Alias	Data Type	Where Used / How Used	Description / Constraints
uploaded_file	image_file	FileStorage (Flask)	POST /predict – received from HTTP request	The raw image file uploaded by the user. Must be PNG, JPG, or JPEG. Max size: 5 MB.
image_grayscale	gray_image	PIL.Image	ImagePreprocessor.convert_to_grayscale()	Grayscale version of the uploaded image. Single-channel representation of pixel intensities.
image_resized	resized_image	PIL.Image	ImagePreprocessor.resize_image()	Image scaled to 28×28 pixels using bicubic resampling to match MNIST training format.
image_array	pixel_array	numpy.ndarray (float32)	ImagePreprocessor.normalize()	Flattened pixel values in range [0, 1]. Shape: (28, 28).

model_input	input_tensor	numpy.ndarray (float32)	CNNModel.predict()	Final model-ready array. Shape: (1, 28, 28, 1). Batch dimension added for Keras compatibility.
probability_output	softmax_output	numpy.ndarray (float32)	CNNModel.predict() return value	Probability distribution across 10 digit classes. Shape: (1, 10). Values sum to 1.0.
predicted_digit	prediction	int (0–9)	DigitRecognitionApp.predict_route()	Final predicted digit class derived from argmax of probability_output.
confidence_score	confidence	float (0.0–1.0)	PredictionResult.to_json()	Probability value for the predicted class. Indicates model certainty.
status	response_status	str	PredictionResult.to_json()	HTTP response status. Values: 'success' or 'error'.
error_message	err_msg	str	InputValidator, Flask error handler	Descriptive message returned to the client when validation or prediction fails.

3.2.2 Normalization

Since the system does not employ a relational database, traditional normalization (1NF, 2NF, 3NF) is not directly applicable. However, the concept of normalization is applied to the input data processed by the system, as described below:

Pixel Value Normalization (Input Data Normalization):

Raw pixel values from the uploaded image exist in the range [0, 255] (8-bit unsigned integers). These are normalized to the range [0.0, 1.0] using the formula:

$$\text{normalized_value} = \text{pixel_value} / 255.0$$

This transformation ensures consistent input scale, prevents numerical instability during model inference, and aligns with the preprocessing applied during CNN training on the MNIST dataset.

Structural Normalization of In-Memory Data:

The in-memory data objects are designed to minimize redundancy:

- The uploaded_file object is never duplicated. It is passed by reference through the preprocessing pipeline.
- The model_input tensor is a single, well-defined structure (shape 1, 28, 28, 1) that is not replicated.
- The PredictionResult object contains only derived fields (predicted_digit, confidence, status), avoiding redundant storage of intermediate processing data.

Note on Future Database Integration:

If the system is extended to include prediction history logging, a relational schema would be introduced. The proposed table (PredictionLog) would adhere to Third Normal Form (3NF): PredictionLog(log_id PK, timestamp, predicted_digit, confidence_score, image_hash). This schema has no partial dependencies (satisfying 2NF) and no transitive dependencies (satisfying 3NF).

3.3 Component Design

3.3.1 Package Diagram

Package Diagram Description

The Package Diagram illustrates the high-level organization of the system into logical packages and their dependencies:

Package: frontend

- Contains: index.html, styles.css
- Responsibility: User interface rendering, form submission, result display.
- Depends on: backend (via HTTP).

Package: backend

- Contains: app.py (Flask application and route definitions)
- Responsibility: Request routing, workflow orchestration.
- Depends on: preprocessing, model, validation packages.

Package: preprocessing

- Contains: preprocessor.py
- Responsibility: Image loading, grayscale conversion, resizing, normalization, reshaping.
- Depends on: PIL (Pillow), numpy packages.

Package: model

- Contains: cnn_model.py, digit_recognition_model.h5
- Responsibility: CNN model loading, inference execution, result extraction.
- Depends on: tensorflow, keras packages.

Package: validation

- Contains: validator.py
- Responsibility: File type and format validation.
- Depends on: Python standard library (os).

Package: external_libraries

- Contains: TensorFlow, Keras, Flask, Pillow, NumPy

- Responsibility: Third-party dependencies providing ML, web, and image processing capabilities.

Dependencies Summary:

- frontend → backend (HTTP)
- backend → preprocessing, model, validation
- preprocessing → external_libraries (Pillow, NumPy)
- model → external_libraries (TensorFlow, Keras)

3.3.2 Component Diagram

Component Diagram Description

The Component Diagram shows the physical software components of the system and their interfaces:

Component	Type	Provided Interface	Required Interface
Web Browser	Client Component	UI Interaction (HTTP)	Flask Server (HTTP POST /predict)
Flask Application (app.py)	Server Component	HTTP API (/index, /predict)	ImagePreprocessor, CNNModel, InputValidator
InputValidator (validator.py)	Utility Component	validate(file): bool	Python os, Flask FileStorage
ImagePreprocessor (preprocessor.py)	Processing Component	preprocess(file): ndarray	Pillow (PIL), NumPy
CNNModel (cnn_model.py)	ML Component	predict(array): int	TensorFlow/Keras, NumPy
Saved Model (digit_model.h5)	Artifact/Data Component	File I/O Interface	HDF5 File System
TensorFlow/Keras Runtime	External Library	Sequential Model API	Hardware (CPU/GPU)

The Flask Application Component acts as the central coordinator: it receives HTTP requests from the Web Browser, delegates file validation to InputValidator, delegates image transformation to ImagePreprocessor, and delegates inference to CNNModel. The CNNModel component loads the Saved Model artifact at startup and uses the TensorFlow runtime for computation.

3.3.3 Deployment Diagram

Deployment Diagram Description

The Deployment Diagram models the physical deployment of software artifacts on hardware nodes for the system:

Node 1: Client Machine (User Device)

- Hardware: Personal computer, laptop, or mobile device.
- Software Artifact: Web Browser (Google Chrome, Mozilla Firefox, Microsoft Edge).
- Communication: Sends HTTP GET (page load) and HTTP POST (image upload) requests to the server.

Node 2: Application Server (Local / Hosted Server)

- Hardware: Standard server or developer workstation with Python 3.8+ environment.
- Operating System: Windows 10 / Ubuntu 20.04 / macOS 12+.
- Deployed Artifacts:
 - – app.py: Flask application entry point (runs on localhost:5000 or deployed host).
 - – validator.py: Input validation module.
 - – preprocessor.py: Image preprocessing module.
 - – cnn_model.py: Model wrapper module.
 - – digit_recognition_model.h5: Pre-trained CNN model file stored on server filesystem.
 - – templates/index.html: HTML template served by Flask.
 - – static/styles.css: CSS stylesheet served as a static file.
- Runtime: Python 3.8+, TensorFlow 2.x, Flask 2.x, Pillow, NumPy.

Communication Protocol:

- Client to Server: HTTP/HTTPS on port 5000 (local) or 80/443 (deployed).
- Inter-module communication: In-process Python function calls.
- Model loading: Local file system read (HDF5 format).

Deployment Modes:

- Development Mode: Flask built-in development server (flask run) on localhost:5000.
- Production Mode: Deployable on cloud platforms (e.g., Heroku, AWS EC2, Google Cloud Run) using a WSGI server such as Gunicorn.

3.4 Interface Design

3.4.1 Screen Shots / Interface Descriptions

Screen 1: Home Page / Upload Interface (index.html)

The home page presents a clean, minimalistic interface consisting of the following elements:

- Page Title: 'Handwritten Digit Recognition' displayed prominently at the top in large bold text, centered on the page.
- Subtitle: 'Upload a handwritten digit image to get an instant prediction' displayed in smaller text below the title.
- File Upload Control: A styled file input field labeled 'Choose File'. Accepts image files (PNG, JPG, JPEG). Displays the selected filename upon selection.

- **Predict Button:** A prominent button labeled 'Predict Digit'. Triggers the HTTP POST request to /predict on click. Styled with a contrasting background color for visibility.
- **Result Display Section:** A clearly labeled area titled 'Prediction Result'. Initially empty; populated with the predicted digit (e.g., 'Predicted Digit: 7') upon successful prediction. Styled with a bordered card component for visual separation.
- **Error Message Area:** Conditionally displayed below the form when an invalid file is uploaded. Shown in red text for user attention (e.g., 'Invalid file type. Please upload a PNG or JPG image.').
- **Color Scheme:** White background, blue accent color (#2E75B6) for headings and buttons, grey for secondary text. Responsive layout supporting desktop and mobile browsers.

Screen 2: Prediction Result Display

Upon successful prediction, the Result Display Section on the same page (index.html) is updated dynamically or via page reload to show:

- **Predicted Digit:** Displayed in large, bold font (e.g., 'Predicted Digit: 3').
- The user may then upload another image without navigating away from the page.

Screen 3: Error Display

If an invalid file is submitted or a server-side error occurs:

- An error notification is rendered on the same page.
- Error text is displayed in red: 'Error: Invalid image file. Please upload a valid PNG or JPG image.'
- The upload form remains accessible for the user to retry.

Appendix C1: Glossary

The following terms and abbreviations are used throughout this Software Design Specification:

Term / Abbreviation	Definition
CNN (Convolutional Neural Network)	A class of deep learning models designed for processing structured grid data such as images. CNNs use convolutional layers to automatically extract spatial features.
MNIST Dataset	Modified National Institute of Standards and Technology dataset. A benchmark dataset of 70,000 grayscale images (60,000 training, 10,000 testing) of handwritten digits (0–9) at 28×28 pixel resolution.
TensorFlow	An open-source machine learning framework developed by Google for building and training neural network models, used in this project for CNN development.
Keras	A high-level neural networks API, written in Python and running on top of TensorFlow, used to define and compile the CNN model architecture.
Flask	A lightweight WSGI micro web framework in Python used to build the backend REST API and serve the web application.
ReLU (Rectified Linear Unit)	An activation function defined as $f(x) = \max(0, x)$. Used in convolutional and dense layers to introduce non-linearity and enable deep network training.
Softmax	An activation function applied in the output layer that converts raw model outputs (logits) into a probability distribution over the 10 digit classes (summing to 1.0).
HDF5 (.h5 file)	Hierarchical Data Format version 5. The file format used by Keras to serialize and save trained model weights and architecture for later inference.
Epoch	One complete pass through the entire training dataset during model training. Multiple epochs are used to iteratively improve model parameters.
Normalization	The process of scaling input pixel values from the range [0, 255] to [0.0, 1.0] to standardize inputs for neural network training and inference.
Inference	The process of using a trained ML model to generate predictions on new, unseen input data without further training.
Overfitting	A condition where a model learns the training data too specifically and performs poorly on unseen test data. Mitigated through techniques such as dropout and data augmentation.
PIL / Pillow	Python Imaging Library (and its modern fork, Pillow). A Python library used for opening, manipulating, and saving image files in various formats.
NumPy	A fundamental Python library for numerical computing. Used to represent and manipulate image data as multi-dimensional arrays (ndarrays) for model input.
MVC (Model-View-Controller)	A software architectural pattern separating an application into three components: Model (data/logic), View (UI), and Controller (request handling). Applied in this system via Flask's routing mechanism.

Appendix C2: To Be Determined (TBD) List

The following items remain to be determined or are planned for future development iterations:

TBD #	Item	Description	Target Resolution
TBD-1	Cloud Deployment Configuration	Specific cloud platform (Heroku, AWS, GCP) and deployment pipeline (CI/CD) for production hosting have not been finalized.	Post-submission, Phase 2
TBD-2	Dataset Expansion Strategy	Methods for expanding the training dataset beyond MNIST to improve real-world generalization (e.g., custom handwritten digit datasets) are under evaluation.	Future Enhancement
TBD-3	Prediction Confidence Threshold	A minimum confidence score threshold below which the model should flag predictions as uncertain has not been defined.	Testing Phase
TBD-4	Image Size Limit Enforcement	Maximum allowable file size for uploads (currently informally set at 5 MB) has not been enforced at the server level.	Backend Development
TBD-5	Prediction History Logging	Whether to implement a database-backed prediction history log for audit or analytics purposes is not yet decided.	Future Enhancement
TBD-6	Mobile Responsiveness	Full testing of the HTML/CSS interface on mobile devices and tablets is pending.	Testing Phase
TBD-7	Model Versioning	A strategy for managing multiple model versions (.h5 files) and switching between them without restarting the server is not yet implemented.	Future Enhancement
TBD-8	API Authentication	Whether the /predict endpoint should require any form of authentication (e.g., API key) for production use has not been determined.	Deployment Phase

End of Software Design Specification Document.